

DOCUMENTAÇÃO TÉCNICA

E-Commerce de Camisetas

Entrega N1 — Documento Unificado

01	Documento de Arquitetura Diagramas C4 · Diagrama de Classes · Design Patterns · Java 21	11 páginas
02	Documentação C4 — Modelo Arquitetural Contexto · Contêineres · Fluxo de Eventos · ADRs · Docker	10 páginas
03	Plano de Qualidade e Testes RNFs · Testes Unitários/Integração/E2E · JaCoCo · Ferramentas	13 páginas

Curso	Análise e Desenvolvimento de Sistemas — 4º Período
Semestre	2026/1
Stack	Java 21 + Spring Boot 3.x + Apache Kafka + Docker
Total	34 páginas (3 documentos + separadores)

Documento de Arquitetura

E-Commerce de Camisetas — Design de Software N1

Conteúdo deste documento

1	Visão Geral da Arquitetura	Stack tecnológico, princípio central de desacoplamento
2	Diagrama C4 — Nível Contexto	Sistema, atores externos e sistemas de terceiros
3	Diagrama C4 — Nível Container	Microserviços, bancos de dados e broker Kafka
4	Diagrama de Classes do Domínio	Entidades, sealed interface, records e Factory
5	Fluxo Assíncrono Kafka	Tópicos, producers, consumers e configuração YAML
6	Design Patterns Aplicados	Factory, Repository, Strategy, Observer, DI
7	Java 21 — Features Utilizadas	Virtual Threads, Records, Sealed Interfaces, Pattern Matching
8	Estrutura dos Repositórios	5 repositórios GitHub + pacotes + critérios de avaliação

DOCUMENTO DE ARQUITETURA

E-Commerce de Camisetas

Diagramas C4 · Diagrama de Classes · Padrões de Projeto

Stack	Java 21 + Spring Boot 3.x + Apache Kafka + Docker
Curso	Análise e Desenvolvimento de Sistemas — 4º Período
Semestre	2026/1

Sumário

1.	Visão Geral da Arquitetura	3
2.	Diagrama C4 — Nível Contexto	4
3.	Diagrama C4 — Nível Container	5
4.	Diagrama de Classes do Domínio	6
5.	Fluxo Assíncrono Kafka	7
6.	Design Patterns Aplicados	8
7.	Java 21 — Features Utilizadas	9
8.	Estrutura dos Repositórios	10

1. Visão Geral da Arquitetura

O projeto é um e-commerce de camisetas construído sobre uma arquitetura de microsserviços. Cada serviço possui responsabilidade única, banco de dados próprio e comunicação assíncrona via Apache Kafka. Um API Gateway centraliza o roteamento. Todo o ambiente é containerizado com Docker Compose.

Princípio central: o order-service não sabe que o notification-service existe. Ele publica um evento no Kafka e qualquer serviço interessado pode consumi-lo. Isso é o desacoplamento exigido pelo documento norteador (0,4 pts na N2).

Stack Tecnológico

Camada	Tecnologia
Backend	Java 21 + Spring Boot 3.x (Spring Web, Data JPA, Kafka, WebSocket)
Mensageria	Apache Kafka 7.5 + Zookeeper (Docker Compose)
Banco de Dados	PostgreSQL 16 (um por serviço) + Redis 7 (cache de status)
Gateway	Spring Cloud Gateway (roteamento, CORS, rate limit)
Frontend	React + Vite (WebSocket via STOMP/SockJS)
Infraestrutura	Docker Compose (todos os containers locais)
Testes	JUnit 5 + Mockito + Testcontainers (PostgreSQL + Kafka)

2. Diagrama C4 — Nível Contexto

O nível Contexto do modelo C4 mostra o sistema como uma caixa preta e seus relacionamentos com os atores externos. Identifica quem usa o sistema e com quais sistemas externos ele se comunica.

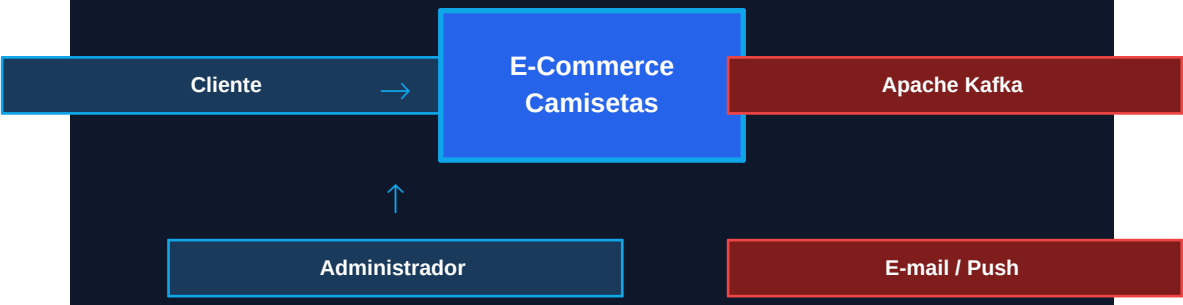


Figura 1 — Diagrama C4 Nível Contexto: sistema e atores externos

Atores e Sistemas Externos

Ator / Sistema	Responsabilidade
Cliente (Comprador)	Pessoa física que navega na vitrine, adiciona ao carrinho e acompanha o pedido em tempo real.
Administrador	Responsável pela loja: cadastra/edita produtos, visualiza pedidos e acessa o dashboard.
Apache Kafka	Broker de mensagens externo ao domínio de negócio. Recebe eventos do order-service e os distribui.
E-mail / Push	Serviço externo de notificação (ex: SendGrid). Acionado pelo notification-service para alertar o cliente.

3. Diagrama C4 — Nível Container

O nível Container detalha os processos em execução dentro do sistema: os microsserviços Spring Boot, o API Gateway, os bancos de dados e o broker Kafka. Cada container é deployável de forma independente.

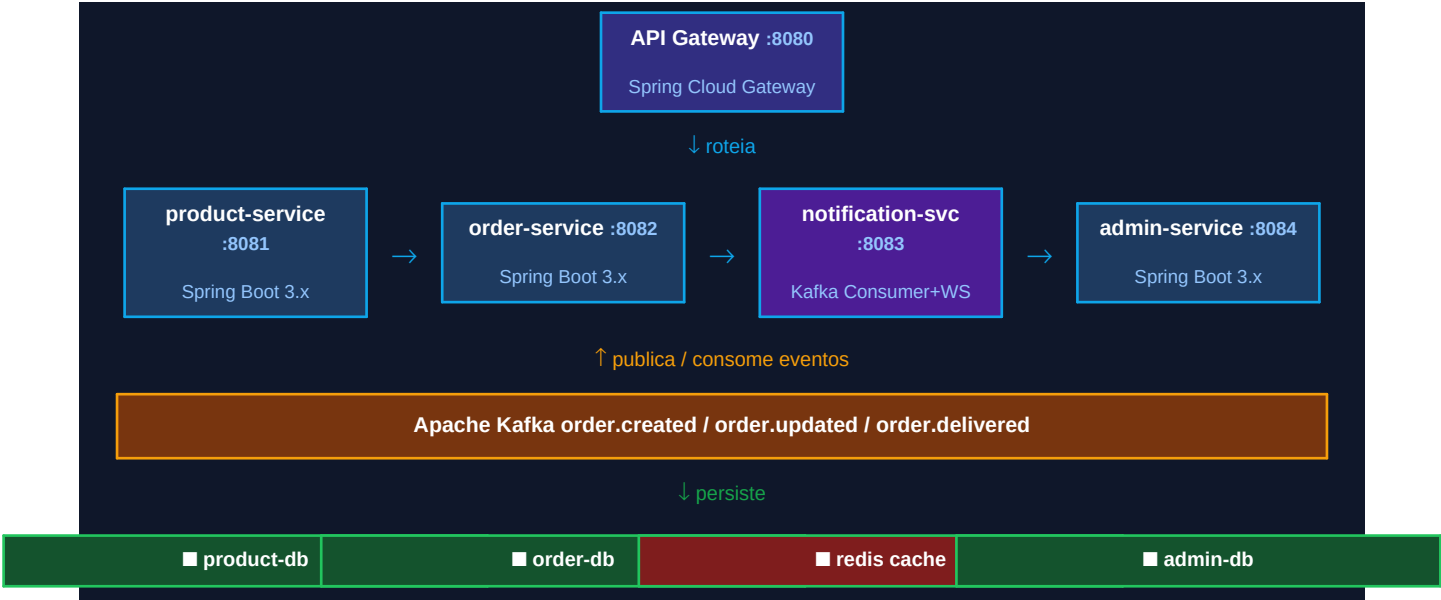


Figura 2 — Diagrama C4 Nível Container: microsserviços, bancos e broker

Descrição dos Containers

Container	Porta	Tecnologia	Responsabilidade
API Gateway	:8080	Spring Cloud Gateway	Ponto de entrada único. Roteia <code>/api/products</code> → <code>product-service</code> , <code>/api/orders</code> → <code>order-service</code> .
product-service	:8081	Spring Boot 3.x	CRUD do catálogo de camisetas. REST puro, sem Kafka. Banco: <code>product-db</code> .
order-service	:8082	Spring Boot 3.x	Cria e gerencia pedidos. Ao criar um pedido publica evento no Kafka. Banco: <code>order-db</code> .
notification-service	:8083	Spring Boot 3.x	Consume eventos Kafka e empurra atualizações ao frontend via WebSocket/STOMP.
admin-service	:8084	Spring Boot 3.x	CRUD de produtos, listagem de pedidos e dashboard de métricas. Banco: <code>admin-db</code> .

4. Diagrama de Classes do Domínio

O diagrama abaixo representa o domínio principal do order-service, que é o núcleo do projeto. Usa recursos modernos do Java 21: sealed interface para modelar a máquina de estados, records imutáveis para DTOs e o padrão Factory para construção do agregado Order.

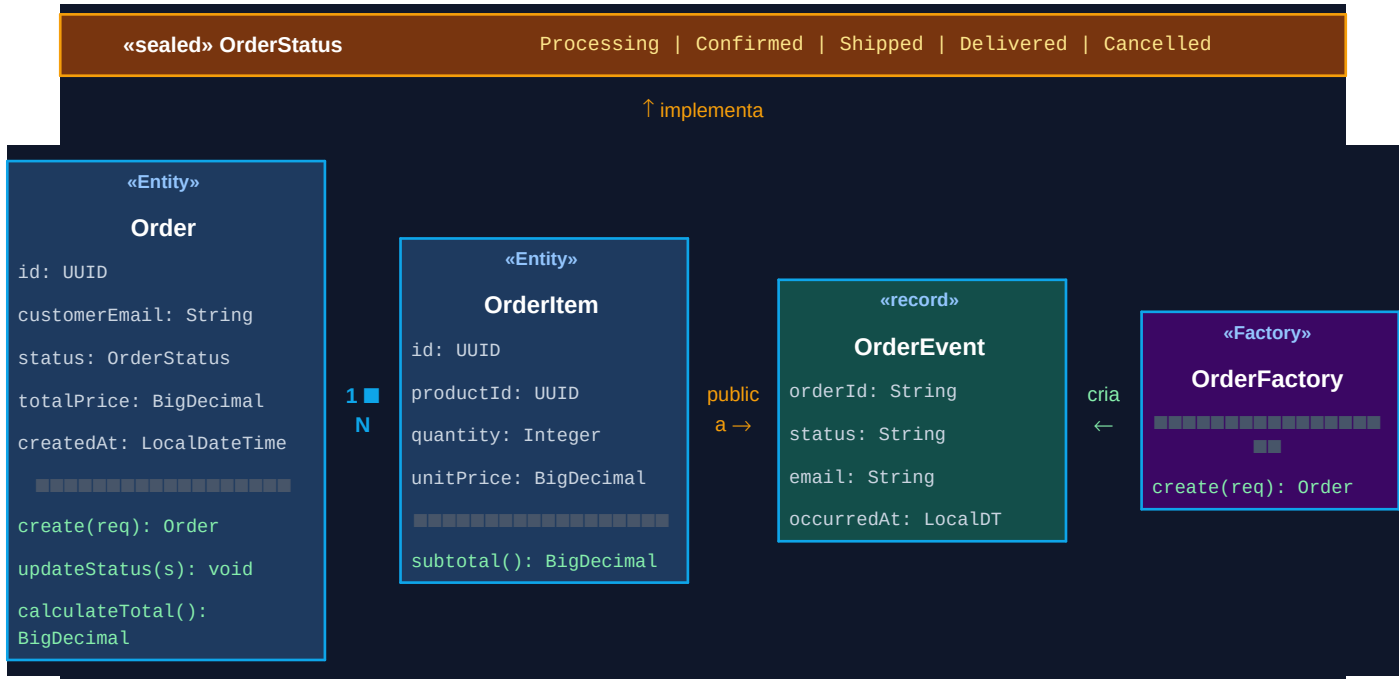


Figura 3 — Diagrama de Classes: domínio do order-service

Detalhamento das Classes

Classe	Tipo	Responsabilidade
Order	Entity	Agregado raiz do domínio. Contém a lista de OrderItems e o estado atual (sealed interface).
OrderItem	Entity	Item individual do pedido: referência ao produto, quantidade e preço unitário. Calcula subtotal.
OrderStatus	sealed interface	Modela a máquina de estados do pedido com tipos fechados: Processing, Confirmed, Shipped, Delivered, Cancelled.
OrderEvent	record	Payload publicado no Kafka. Imutável por ser record Java 21. Contém orderId, status, email.
OrderFactory	Factory	Centraliza a lógica de criação do agregado Order a partir de um OrderRequest. Aplica validações e calcula total.

Exemplo — Sealed Interface (Java 21)

```
public sealed interface OrderStatus
    permits Processing, Confirmed, Shipped, Delivered, Cancelled {}

// Pattern Matching – o compilador garante todos os casos:
String label = switch (status) {
    case Processing p -> "Processando...";
    case Confirmed c -> "Pedido confirmado!";
```



```
case Shipped s -> "A caminho!";  
case Delivered d -> "Entregue!";  
case Cancelled c -> "Cancelado.";  
};
```

5. Fluxo Assíncrono — Apache Kafka

Este é o fluxo mais importante do projeto e o diferencial técnico exigido pelo documento norteador. O cliente finaliza o pedido, o order-service persiste e publica um evento — sem esperar resposta do notification-service. O frontend é atualizado em tempo real via WebSocket sem recarregar a página.

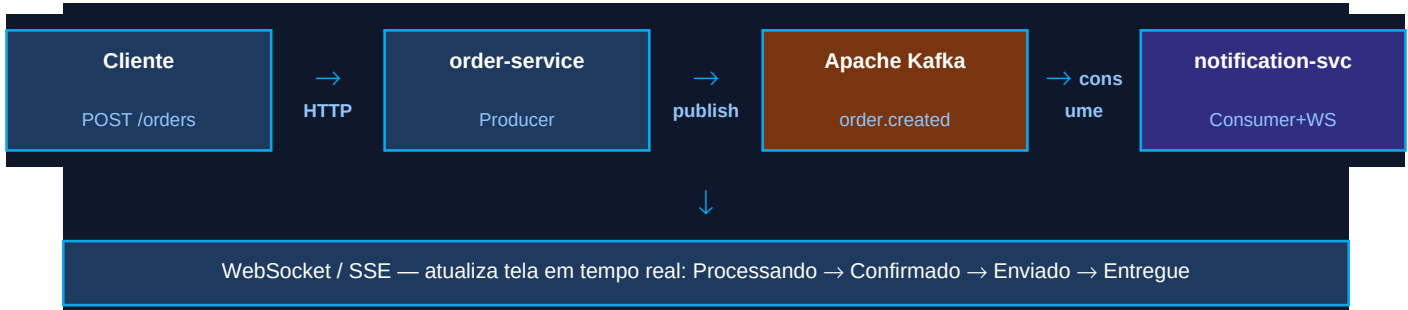


Figura 4 — Fluxo assíncrono: do pedido à atualização da tela em tempo real

Tópicos Kafka

Tópico	Producer	Consumer(s)	Quando é publicado
order.created	order-service	notification-service	Cliente finaliza o carrinho
order.updated	order-service	notification-service	Admin atualiza o status manualmente
order.delivered	order-service	notification-service, admin-service	Pedido marcado como entregue

Configuração Spring Kafka (application.yml)

```
spring:
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      value-serializer: JsonSerializer
      acks: all # garante persistência no broker
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      value-deserializer: JsonDeserializer
    properties:
      spring.json.trusted.packages: "com.ecommerce.*"
```

6. Design Patterns Aplicados

Os padrões de projeto foram escolhidos para atender o critério de Qualidade de Código (0,5 pts na N2). Cada padrão resolve um problema concreto do projeto, não foi aplicado apenas para cumprir o requisito.

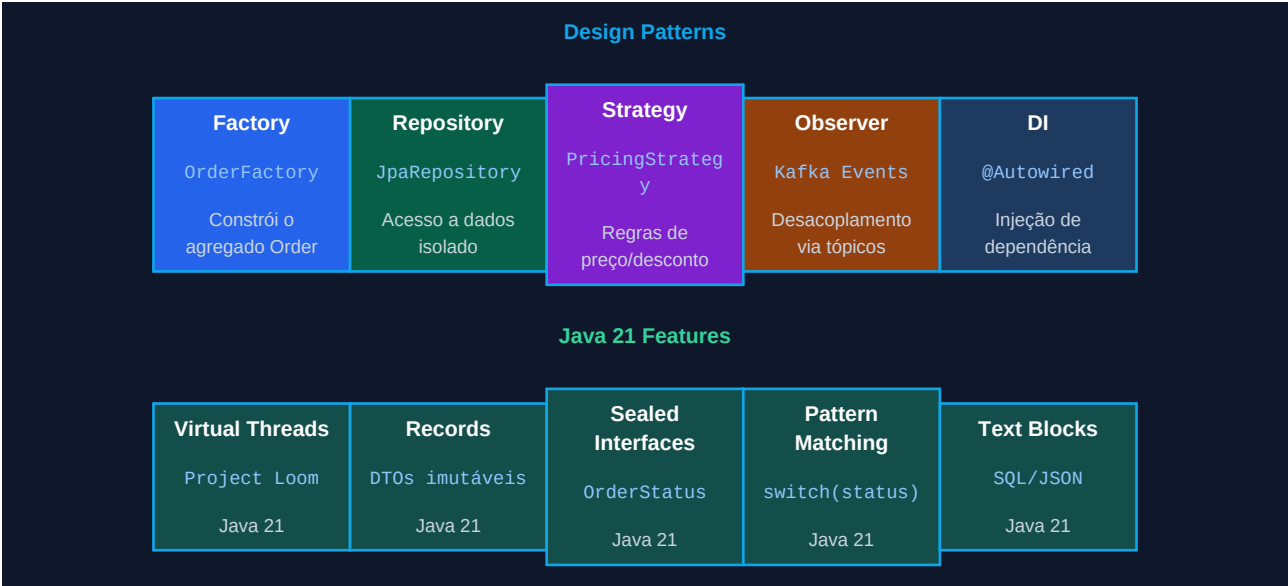


Figura 5 — Design Patterns e features Java 21 utilizados no projeto

Pattern	Implementação	Onde	Justificativa
Factory	OrderFactory	order-service	Centraliza a criação do agregado Order a partir de um OrderRequest.
Repository	JpaRepository	Todos os services	Separa a camada de acesso a dados da lógica de negócio. Facilita mocking.
Strategy	PricingStrategy (interface)	order-service	Encapsula regras de precificação/desconto. Novas regras não alteram o código existente.
Observer	Kafka Topics + @KafkaListener	order / notification	O order-service publica eventos sem conhecer os consumidores. Desacoplamento total.
DTO	Java Records	Controllers	Records imutáveis como DTOs de entrada/saída. Thread-safe por natureza.
DI	@Autowired / Construtor	Todo o projeto	Inversão de controle nativa do Spring. Facilita mocking nos testes unitários.

7. Java 21 — Features Utilizadas

O uso das features modernas do Java 21 demonstra domínio tecnológico e é um argumento técnico forte para a defesa do projeto na banca.

Feature	Onde usar	Benefício
Virtual Threads (Project Loom)	application.yml: spring.threads.virtual.enabled=true	Substitui pool de threads por threads virtuais leves. O servidor suporta muito mais requisições simultâneas com menos memória.
Records	OrderRequest, OrderResponse, OrderEvent, ProductResponse	DTOs imutáveis com zero boilerplate. Construtores, getters, equals, hashCode e toString gerados pelo compilador.
Sealed Interfaces	OrderStatus permits Processing, Confirmed...	Fecha a hierarquia de tipos. O compilador garante que todos os estados são tratados no pattern matching.
Pattern Matching (switch)	switch(status) { case Shipped s -> ...}	Desestrutura e acessa o tipo sem casting manual. Código mais expressivo e seguro.
Text Blocks	SQL nativo em repositórios @Query	Strings multilinhas legíveis para queries SQL complexas e payloads JSON.

Para a banca: habilitar Virtual Threads é uma linha de configuração, mas demonstra conhecimento do ecossistema Java 21. Mencionar que é o Project Loom e que resolve o problema de thread-per-request do modelo bloqueante tradicional gera destaque na arguição.

8. Estrutura dos Repositórios GitHub

O projeto utiliza 5 repositórios na organização GitHub, seguindo o padrão de um repositório por microsserviço (exceto o Backend que agrupa product, order e admin como módulos Maven).

Repositório	Conteúdo	Tipo
Projeto-Integrador-Infra	docker-compose.yml, .env.example, README geral da organização	Infra
Projeto-Integrador-Backend	Módulos Maven: product-service, admin-service (Spring Boot 3.x + JPA)	Backend
Projeto-Integrador-Logistics-Service	order-service: criação de pedidos + Kafka Producer	Backend
Projeto-Integrador-Notification-Service	Kafka Consumer + WebSocket/SSE: notificações em tempo real	Backend
Projeto-Integrador-Frontend	React + Vite: vitrine, produto, carrinho, status do pedido, painel admin	Frontend

Estrutura de Pacotes — Padrão por Serviço Spring Boot

```
com.ecommerce.{servico}/  
  
■■■ controller/ ← @RestController + DTOs de entrada/saída  
  
■■■ service/ ← @Service + lógica de negócio + Strategy  
  
■■■ repository/ ← JpaRepository (acesso a dados)  
  
■■■ messaging/ ← KafkaTemplate (producer) / @KafkaListener (consumer)  
  
■■■ domain/ ← Entidades, Sealed Interface, Factory – zero framework  
  
■■■ dto/ ← Java Records (Request / Response / Event)
```

Regra de ouro da Clean Architecture: o pacote domain/ não pode importar nada do Spring Framework. As entidades e regras de negócio devem ser puro Java, facilitando testes unitários sem subir o contexto Spring (mais rápido, mais simples).

Critérios de Avaliação × Arquitetura

Critério	Como este projeto atende
Coerência da Arquitetura (0,3 N1)	Diagramas C4 + estrutura de microsserviços documentada neste documento.
Mensageria desacoplada (0,4 N2)	order-service publica no Kafka sem conhecer o notification-service.
Implementação funcionando (0,4 N2)	Fluxo completo: pedido → Kafka → WebSocket → atualização na tela.
Design Patterns + Arq. Limpa (0,5 N2)	Factory, Repository, Strategy, Observer (Kafka), DI — todos documentados.
Testes (0,4 N2)	JUnit + Mockito (unitários) + Testcontainers (integração com Kafka/PostgreSQL).

Documentação C4

Modelo Arquitetural — Contexto e Contêineres

Conteúdo deste documento

1	Introdução e Visão Geral	Objetivos do sistema e stack tecnológica justificada
2	Nível 1 — Diagrama de Contexto (C1)	Atores externos: Cliente, Admin, E-mail, Gateway de Pagamento
3	Nível 2 — Diagrama de Contêineres (C2)	Frontend, Backend, Logistics, Notification, Kafka, PG, Redis
4	Fluxo de Eventos — Ciclo de Vida de um Pedido	7 passos do POST /orders até a atualização WebSocket
5	Estrutura de Repositórios	5 repositórios organizados na org GitHub do grupo
6	Justificativa Kafka vs REST Síncrono	Comparativo de acoplamento, disponibilidade e UX
7	Decisões Arquiteturais (ADRs)	ADR-01 Microsserviços · ADR-02 Kafka · ADR-03 WebSocket
8/9	Infraestrutura Docker Compose	Serviços, portas, depends_on e configurações
10	Aderência aos Critérios de Avaliação N1	Mapeamento de critérios para artefatos entregues

PROJETO INTEGRADOR

Análise e Desenvolvimento de Sistemas — 4º Semestre

Documentação de Arquitetura

Modelo C4 — Contexto e Contêineres

Solução Distribuída para Logística e Rastreamento de Entregas

2026/1

1. Introdução e Visão Geral

Este documento descreve a arquitetura do sistema de logística e rastreamento de entregas desenvolvido como Projeto Integrador do curso de Análise e Desenvolvimento de Sistemas (4º Semestre, 2026/1). A documentação segue o modelo C4, abrangendo o Diagrama de Contexto (nível 1) e o Diagrama de Contêineres (nível 2).

O sistema atende ao seguinte problema central: o crescimento do e-commerce exige plataformas capazes de processar pedidos, notificar status em tempo real e gerenciar falhas de forma resiliente — sem bloquear a experiência do usuário durante o processamento assíncrono.

Objetivos do Sistema

- Permitir que clientes realizem pedidos de produtos (camisetas) por meio de interface web.
- Processar pedidos de forma assíncrona utilizando Apache Kafka como broker de mensageria.
- Atualizar o status do pedido em tempo real no navegador via WebSocket, sem polling.
- Garantir desacoplamento entre os serviços de backend por meio de eventos de domínio.
- Orquestrar toda a infraestrutura com Docker Compose, facilitando deploy e reprodutibilidade.

Stack Tecnológica

Camada	Tecnologia / Justificativa
Backend	Java 21 + Spring Boot 3.x — runtime moderno com suporte a Virtual Threads e ecossistema robusto para microserviços.
Mensageria	Apache Kafka — broker distribuído de alta throughput; desacopla produtores e consumidores; permite replay e auditoria de eventos.
Banco de Dados	PostgreSQL — RDBMS confiável para persistência de pedidos, usuários e catálogo.
Cache	Redis — cache de sessão e filas de notificação temporárias.
Frontend	React + Vite — SPA com atualização reativa via SockJS/STOMP para exibição de status em tempo real.
Infraestrutura	Docker Compose — orquestra todos os serviços em containers isolados com rede interna compartilhada.

2. Nível 1 — Diagrama de Contexto (C1)

O Diagrama de Contexto apresenta o sistema como uma caixa preta, identificando todos os atores externos (pessoas e sistemas) que interagem com ele. É o nível mais alto de abstração do modelo C4.



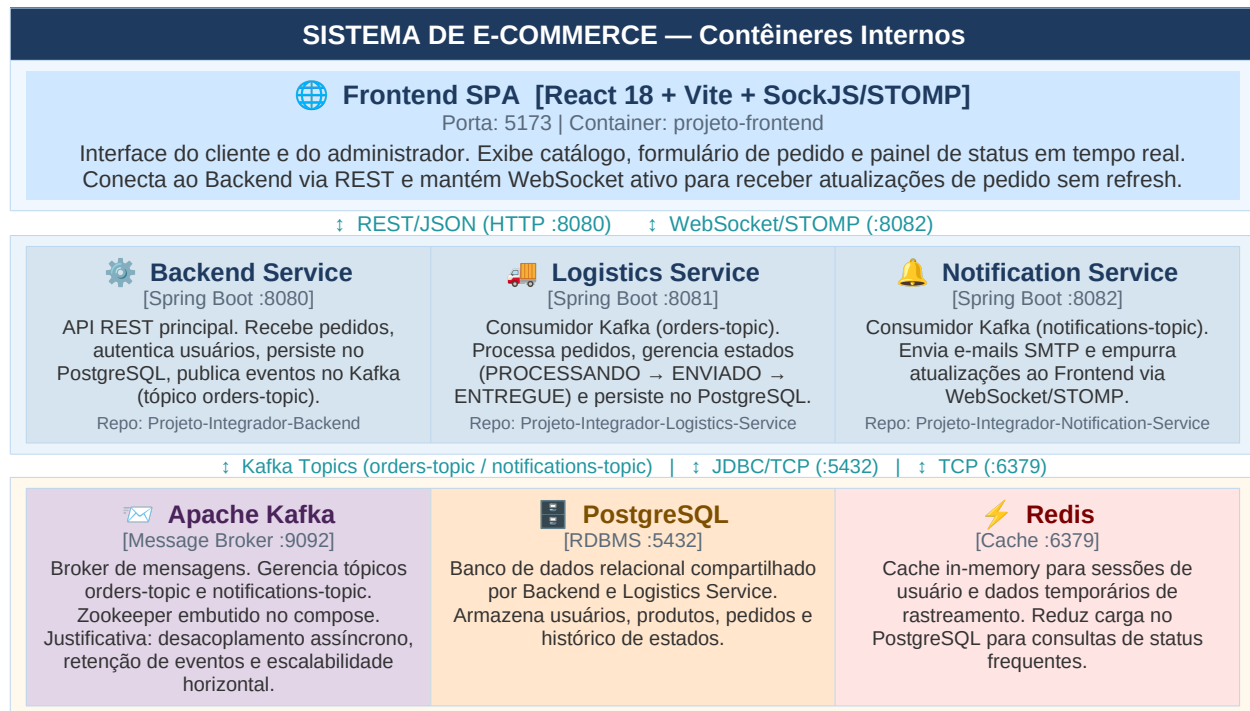
Atores e Sistemas Externos

Elemento	Tipo	Descrição
Cliente	Pessoa	Usuário final que navega pelo catálogo, realiza compras e acompanha o status de entrega em tempo real via interface web.
Administrador	Pessoa	Operador interno que gerencia catálogo de produtos, estoque, pedidos e monitora a saúde do sistema via dashboard.
Serviço de E-mail	Sistema Externo	Provedor SMTP (ex: SendGrid) acionado pelo Notification Service para envio de e-mails transacionais.
Gateway de Pagamento	Sistema Externo	API de terceiro (ex: Stripe) para autorização e captura de pagamentos. Integrado ao Backend Service via REST.

3. Nível 2 — Diagrama de Contêineres (C2)

O Diagrama de Contêineres detalha o interior do sistema, mostrando os contêineres implantáveis (aplicações, bancos de dados, message brokers) e como eles se comunicam. Cada contêiner representa um processo separado que pode ser implantado e escalado de forma independente.

[DIAGRAMA DE CONTÊINERES – C2]



Descrição dos Contêineres

Contêiner	Tecnologia	Porta	Responsabilidade Principal
Frontend SPA	React + Vite	5173	Interface do cliente/admin. Renderiza catálogo, envia pedidos via REST e exibe atualizações de status em tempo real via WebSocket/STOMP.
Backend Service	Spring Boot 3.x	8080	API REST principal. CRUD de produtos, usuários e pedidos. Publica eventos OrderCreatedEvent no tópico orders-topic do Kafka.
Logistics Service	Spring Boot 3.x	8081	Microserviço de pedidos. Consome orders-topic, gerencia a máquina de estados do pedido (PROCESSANDO → ENVIADO → ENTREGUE) e persiste transições.
Notification Service	Spring Boot 3.x	8082	Consome notifications-topic. Envia e-mails transacionais via SMTP e empurra eventos de status ao Frontend conectado via WebSocket/STOMP.
Apache Kafka	Kafka + Zookeeper	9092	Message broker. Gerencia os tópicos orders-topic (pedidos) e notifications-topic (alertas). Garante entrega assíncrona e desacoplamento entre serviços.
PostgreSQL	PostgreSQL 15	5432	Banco de dados relacional. Armazena domínios: usuários, produtos, pedidos e histórico de transições de estado.

Redis	Redis 7	6379	Cache in-memory para sessões de usuário autenticado e dados de rastreamento frequentemente consultados.
--------------	---------	------	---

4. Fluxo de Eventos — Ciclo de Vida de um Pedido

O fluxo abaixo descreve o caminho completo de um pedido, desde a ação do cliente até a atualização em tempo real na interface. Este é o fluxo central que demonstra o uso da mensageria assíncrona como diferencial arquitetural.

#	Ator / Componente	Ação
1	Cliente (Browser)	Preenche formulário de pedido e clica em Confirmar. O Frontend envia POST /orders para o Backend Service via REST/JSON.
2	Backend Service	Valida a requisição, persiste o pedido no PostgreSQL com status PENDENTE e publica o evento OrderCreatedEvent no tópico Kafka orders-topic.
3	Apache Kafka	Recebe a mensagem no tópico orders-topic e a mantém disponível para consumidores subscritos. A resposta ao cliente (HTTP 202 Accepted) já foi enviada — sem bloqueio.
4	Logistics Service	Consome a mensagem de orders-topic, processa a lógica de logística e atualiza o estado do pedido para PROCESSANDO. Publica evento de status no tópico notifications-topic.
5	Notification Service	Consome notifications-topic. Envia e-mail de confirmação ao cliente (SMTP) e emite atualização via WebSocket STOMP no destino /topic/orders/{id}.
6	Frontend (WebSocket)	O browser, com conexão WebSocket ativa, recebe o evento e atualiza o badge de status do pedido em tempo real sem recarregamento de página.
7	Logistics Service	Simula progressão: após intervalo, publica novos eventos (ENVIADO, ENTREGUE) que repetem os passos 5 e 6 até o estado final.

Tópicos Kafka

Tópico	Partições	Produtor	Consumidor(es)
orders-topic	1 (dev) / 3 (prod)	Backend Service	Logistics Service
notifications-topic	1 (dev) / 3 (prod)	Logistics Service	Notification Service

5. Estrutura de Repositórios

O projeto é organizado em múltiplos repositórios sob uma organização GitHub do grupo, seguindo o princípio de separação de responsabilidades por serviço.

Repositório	Tipo	Conteúdo Principal
Projeto-Integrador-Infra	Infraestrutura	docker-compose.yml master, arquivos .env, configurações de rede e volumes Docker.
Projeto-Integrador-Backend	Microserviço	API REST principal (Spring Boot). CRUD, autenticação, produtor Kafka.
Projeto-Integrador-Logistics-Service	Microserviço	Consumidor Kafka orders-topic. Máquina de estados do pedido. Produtor notifications-topic.
Projeto-Integrador-Notification-Service	Microserviço	Consumidor Kafka notifications-topic. WebSocket STOMP. SMTP.
Projeto-Integrador-Frontend	Frontend	SPA React + Vite. Interface do cliente e admin. SockJS/STOMP para tempo real.

6. Justificativa Arquitetural — Kafka vs. REST Síncrono

Um dos requisitos acadêmicos mais relevantes deste projeto é o uso obrigatório de mensageria. A tabela abaixo justifica a escolha do Apache Kafka em detrimento de chamadas REST diretas entre serviços.

Critério	REST Síncrono	Apache Kafka (escolha)
Acoplamento	Alto — o produtor precisa conhecer o endpoint do consumidor.	Baixo — produtor publica no tópico; consumidores são independentes.
Disponibilidade	Se o consumidor cair, o produtor recebe erro imediato.	Mensagens ficam retidas no broker até o consumidor subir novamente.
Escalabilidade	Cada nova integração exige novo endpoint e versionamento.	Novos consumidores se inscrevem no tópico sem alterar o produtor.
Rastreabilidade	Sem histórico de chamadas nativo.	Eventos são persistidos e podem ser reprocessados (replay).
Desempenho	Bloqueante — o cliente espera cada etapa concluir.	Não-bloqueante — Backend retorna 202 imediatamente após publicar.
UX (tempo real)	Requer polling periódico do frontend para verificar mudanças.	WebSocket empurra atualizações ao browser sem polling.

7. Decisões Arquiteturais (ADRs Resumidas)

ADR-01 — Arquitetura de Microserviços

Contexto: O sistema precisa evoluir de forma independente em cada domínio (catálogo, pedidos, notificações).

Decisão: Adotar microserviços em vez de monolito.

Consequências:

- (+) Cada serviço pode ser deployado, escalado e versionado de forma independente.
- (+) Falhas em um serviço não derrubam os demais.
- (–) Maior complexidade operacional — mitigada com Docker Compose.

ADR-02 — Comunicação Assíncrona via Kafka

Contexto: O processamento de pedidos não deve bloquear a resposta ao cliente.

Decisão: Usar Apache Kafka como broker de eventos entre Backend, Logistics e Notification Service.

Consequências:

- (+) Desacoplamento temporal — serviços não precisam estar disponíveis simultaneamente.
- (+) Rastreabilidade — eventos persistidos e reprocessáveis.
- (–) Overhead de configuração — aceito dado o contexto educacional da disciplina de Mensageria.

ADR-03 — WebSocket/STOMP para Notificações em Tempo Real

Contexto: O frontend precisa exibir mudanças de status sem polling.

Decisão: Usar Spring WebSocket + STOMP no Notification Service; SockJS no React.

Consequências:

- (+) UX fluida — atualizações chegam em < 1s sem recarregar a página.
- (+) Menor tráfego de rede comparado a polling a cada 5s.
- (–) Conexão persistente por usuário — aceitável para escala do projeto.

8. Infraestrutura — Docker Compose

Toda a infraestrutura é orquestrada via docker-compose.yml no repositório Projeto-Integrador-Infra. Os serviços compartilham a rede interna ecommerce-network.

Serviço Docker	Porta Host	Porta Container	Observações
zookeeper	2181	2181	Coordenação do cluster Kafka. Embutido no Compose para simplicidade.
kafka	9092	9092	Broker principal. KAFKA_ADVERTISED_LISTENERS aponta para host.docker.internal.
postgres	5432	5432	Volume nomeado para persistência entre restarts do container.
redis	6379	6379	Modo standalone. Sem autenticação em dev.
backend	8080	8080	depends_on: postgres, kafka. Spring Profile: docker.

logistics-service	8081	8081	depends_on: kafka, postgres. Consumer Group: logistics-group.
notification-service	8082	8082	depends_on: kafka. WebSocket endpoint: /ws.
frontend	5173	5173	Vite dev server. VITE_API_URL=http://localhost:8080.

9. Aderência aos Critérios de Avaliação (N1)

A tabela abaixo mapeia os critérios de avaliação da N1 para os artefatos entregues neste documento e nos demais repositórios do projeto.

Critério (N1)	Pontos	Evidência / Artefato
Coerência da Arquitetura	0,3	Diagrama C4 Contexto (Seção 2) e Contêineres (Seção 3). Justificativa Kafka vs REST (Seção 6). ADRs (Seção 7).
Qualidade da Interface	0,3	Protótipos Figma (entregue pela equipe de UI). Este documento especifica fluxos que guiam o design.
Documentação	0,2	Este documento C4 + Plano de Testes (documento separado). Diagramas claros com descrição de cada elemento.
Organização / Git	0,2	5 repositórios organizados na org GitHub do grupo. Hello World Kafka → WebSocket → Frontend já executado com sucesso.

Análise e Desenvolvimento de Sistemas | 2026/1
Documentação gerada para entrega N1 — Projeto Integrador

Plano de Qualidade e Testes

E-Commerce de Camisetas — Projeto Integrador N1

Conteúdo deste documento

1	Introdução e Objetivos de Qualidade	Escopo dos testes e tipos adotados
2	Requisitos Não-Funcionais (RNF)	14 RNFs: Desempenho, Disponibilidade, Segurança, Manutenibilidade, Portabilidade
3	Plano de Testes — Visão Geral	Pirâmide de testes: unitário, integração, E2E, não-funcional
4	Cenários de Teste — Unitários	13 cenários (CT-U-01 a CT-U-13) com JUnit 5 + Mockito
5	Cenários de Teste — Integração	6 cenários com Testcontainers (PostgreSQL + Kafka reais)
6	Cenários de Teste — End-to-End	5 cenários incluindo CT-E-03 para demonstração na banca
7	Cenários de Teste — Não-Funcionais	Desempenho (k6/JMeter), Segurança (cURL), Portabilidade
8	Ferramentas e Configuração	JUnit, Mockito, Testcontainers, JaCoCo, REST Assured, k6
9	Critérios de Aceite e Cobertura	Rastreabilidade RNF × Critério de Avaliação N2

PLANO DE QUALIDADE

E-Commerce de Camisetas

Plano de Testes e Requisitos Não-Funcionais

Stack	Java 21 + Spring Boot 3.x + Apache Kafka + Docker
Curso	Análise e Desenvolvimento de Sistemas — 4º Período
Semestre	2026/1

Sumário

1.	Introdução e Objetivos de Qualidade	3
2.	Requisitos Não-Funcionais (RNF)	4
3.	Plano de Testes — Visão Geral	6
4.	Cenários de Teste — Unitários	7
5.	Cenários de Teste — Integração	9
6.	Cenários de Teste — End-to-End	10
7.	Cenários de Teste — Não-Funcionais	11
8.	Ferramentas e Configuração	12
9.	Critérios de Aceite e Cobertura	13

1. Introdução e Objetivos de Qualidade

Este documento define o Plano de Testes e os Requisitos Não-Funcionais do e-commerce de camisetas desenvolvido como Projeto Integrador do curso de ADS. O sistema é composto por microsserviços Spring Boot comunicando-se de forma assíncrona via Apache Kafka, com frontend React recebendo atualizações em tempo real via WebSocket.

Objetivo principal: garantir que o fluxo assíncrono Pedido → Kafka → Notificação funcione de forma confiável, desacoplada e com tempo de resposta satisfatório, atendendo os critérios de avaliação da N2 (0,4 pts mensageria + 0,4 pts testes).

1.1 Escopo dos Testes

Serviço	O que será testado
product-service	CRUD de produtos, validações de entrada, persistência
logistics-service	Criação de pedidos, publicação de eventos Kafka, máquina de estados
notification-service	Consumo de eventos Kafka, entrega via WebSocket
admin-service	Listagem de pedidos, métricas do dashboard
Infra / Mensageria	Kafka Producer/Consumer, reconexão, tolerância a falhas

1.2 Tipos de Teste Adotados

Tipo	Descrição
Unitário	Testa cada classe isolada com mocks. Rápido, sem dependências externas.
Integração	Testa a comunicação entre camadas com banco e Kafka reais via Testcontainers.
End-to-End	Valida o fluxo completo do sistema, do frontend ao banco.
Não-Funcional	Valida desempenho, segurança, disponibilidade e manutenibilidade.

2. Requisitos Não-Funcionais (RNF)

Os Requisitos Não-Funcionais definem as características de qualidade do sistema. Cada RNF possui critério de aceite mensurável para validação objetiva na banca.

2.1 Desempenho

ID	Nome	Critério de Aceite	Como Validar
RNF-01	Tempo de resposta API	Endpoints REST respondem em até 500ms para 95% das requisições em carga normal.	Teste de carga com 50 usuários simultâneos por 60 segundos.
RNF-02	Latência do fluxo Kafka	O evento publicado no Kafka deve ser consumido e entregue via WebSocket em até 2 segundos.	Medir timestamp de publicação vs. recebimento no frontend.
RNF-03	Throughput mínimo	O sistema deve processar no mínimo 20 pedidos por segundo sem degradação.	Teste de carga com Apache JMeter ou k6.

2.2 Disponibilidade e Confiabilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-04	Disponibilidade	O sistema deve estar disponível 99% do tempo durante o período de avaliação.	Monitorar com Spring Actuator /health durante as demos.
RNF-05	Tolerância a falhas Kafka	Se o notification-service reiniciar, ele deve recuperar os eventos perdidos ao voltar.	Derrubar o container, publicar eventos, subir novamente e verificar consumo.
RNF-06	Persistência de dados	Pedidos criados devem persistir mesmo após restart dos serviços.	Criar pedido, reiniciar o container order-db, consultar o pedido.

2.3 Segurança

ID	Nome	Critério de Aceite	Como Validar
RNF-07	Proteção de credenciais	Nenhuma senha, chave ou segredo deve estar hardcoded no código-fonte ou subir ao GitHub.	Revisar todos os commits. Credenciais apenas no .env (não versionado).
RNF-08	CORS configurado	O backend deve rejeitar requisições de origens não autorizadas.	Fazer requisição de origem diferente e verificar resposta 403.
RNF-09	Validação de entrada	Dados inválidos enviados à API devem retornar HTTP 400 com mensagem descritiva.	Enviar payload sem campos obrigatórios e verificar resposta.

2.4 Manutenibilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-10	Cobertura de testes	Cobertura mínima de 70% nas classes da camada service de cada microserviço.	Relatório gerado pelo JaCoCo via Maven: mvn verify.
RNF-11	Separação de responsabilidades	O pacote domain/ não deve importar nenhuma classe do Spring Framework.	Revisar imports. Zero dependências de framework no domínio.
RNF-12	Configuração externalizada	Toda configuração de ambiente deve estar em variáveis de ambiente (.env), nunca no código.	Trocar valor do .env e verificar que o sistema adota sem recompilar.

2.5 Portabilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-13	Containerização	Todo o ambiente deve subir com um único comando: docker compose up -d.	Clonar o repositório em máquina limpa e executar o comando.
RNF-14	Independência de SO	O projeto deve funcionar em Windows, macOS e Linux sem alterações no código.	Testar em ao menos dois sistemas operacionais distintos.

3. Plano de Testes — Visão Geral

A estratégia de testes segue a pirâmide de testes: maior quantidade de testes unitários (rápidos e baratos), quantidade menor de testes de integração, e poucos testes end-to-end (lentos mas de alto valor para a banca).

Tipo	Ferramentas	O que cobre	Qtd. est.	Duração	Quando rodar
Unitário	JUnit 5 + Mockito	Service, Factory, Domain	~50	< 10s	A cada commit
Integração	Spring Boot Test + Testcontainers	Repository, Kafka, REST	~20	< 60s	A cada PR
End-to-End	REST Assured + Testcontainers	Fluxo completo	~10	< 3min	Antes da entrega
Não-Funcional	JMeter / k6	Carga e latência	~5	~5min	Antes da banca

Para a N2: o que o professor vai querer ver é o relatório do JaCoCo mostrando cobertura $\geq 70\%$ na camada service, e pelo menos um teste de integração com Testcontainers provando que o Kafka funciona de ponta a ponta.

4. Cenários de Teste — Unitários

Testam uma única classe em isolamento. Todas as dependências externas (banco, Kafka, outros serviços) são substituídas por mocks com Mockito.

4.1 product-service

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-01	Listar produtos ativos	Repositório retorna lista com 3 produtos ativos	Service retorna lista com 3 ProductResponse	PASS
CT-U-02	Buscar produto por ID inexistente	Repositório retorna Optional.empty()	Service lança ResourceNotFoundException com HTTP 404	PASS
CT-U-03	Criar produto com dados válidos	Request com nome, preço e estoque preenchidos	Repositório chamado 1x; retorna ProductResponse com ID	PASS
CT-U-04	Criar produto com preço negativo	Request com price = -10.00	Lança ConstraintViolationException antes de chamar repositório	PASS
CT-U-05	Deletar produto — soft delete	Produto existe no repositório	Campo active = false; repositório.save() chamado 1x	PASS

4.2 logistics-service (order-service)

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-06	Criar pedido com itens válidos	OrderRequest com 2 itens e email válido	Order salvo com status Processing; KafkaTemplate.send() chamado 1x	PASS
CT-U-07	Criar pedido sem itens	OrderRequest com lista de itens vazia	Lança IllegalArgumentException; repositório NÃO é chamado	PASS
CT-U-08	Atualizar status para Shipped	Pedido existente com status Confirmed	Status atualizado para Shipped; evento order.updated publicado	PASS
CT-U-09	OrderFactory — construção do agregado	OrderRequest com 3 itens	Order criado com totalPrice = soma dos itens; status = Processing	PASS
CT-U-10	Sealed interface — pattern matching	OrderStatus = Delivered	switch retorna label 'Entregue!' sem lançar exceção	PASS
CT-U-11	Calcular total do pedido	2 itens: R\$49,90 x2 e R\$79,90 x1	totalPrice = R\$179,70	PASS

4.3 notification-service

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-12	Consumir evento e enviar WebSocket	OrderEvent com orderId e status 'Shipped'	messagingTemplate.convertAndSend() chamado com /topic/orders	PASS
CT-U-13	Consumir evento com payload nulo	Mensagem Kafka com body null	Lança exceção controlada; não trava o consumer	PASS

Exemplo de código — CT-U-06:

```
when(orderRepository.save(any())).thenReturn(savedOrder);  
when(productClient.findById(any())).thenReturn(productResponse);  
orderService.create(request);
```



```
verify(kafkaTemplate, times(1)).send(eq("order.created"), any());
```

5. Cenários de Teste — Integração

Usam Testcontainers para subir PostgreSQL e Kafka reais em Docker durante a execução dos testes. Validam a comunicação real entre as camadas do sistema.

Por que Testcontainers? Garante que o teste roda com as mesmas versões do banco e do Kafka que rodam em produção. Elimina o 'funciona na minha máquina'.

ID	Cenário	Entrada	Resultado Esperado	Infra
CT-I-01	Criar pedido e verificar persistência	POST /api/orders com payload válido	HTTP 201; pedido encontrado no PostgreSQL; status = PROCESSING	Testcontainers PG
CT-I-02	Publicar evento no Kafka e consumir	OrderServiceImpl.create() chamado	Evento encontrado no tópico order.created em até 5s	Testcontainers Kafka
CT-I-03	Fluxo Kafka completo no teste	Producer publica no tópico hello.world	Consumer recebe a mensagem; contador de mensagens = 1	Testcontainers Kafka
CT-I-04	Buscar produto pelo endpoint REST	GET /api/products/{id} com produto no banco	HTTP 200 com JSON contendo todos os campos do produto	Testcontainers PG
CT-I-05	Validação de entrada — endpoint REST	POST /api/orders com body vazio {}	HTTP 400 com lista de erros de validação	Testcontainers PG
CT-I-06	Reconexão do consumer após restart	Kafka reinicia; producer envia mensagem; consumer volta	Consumer recebe a mensagem pendente (auto-offset-reset=earliest)	Testcontainers Kafka

5.1 Configuração Testcontainers

```
@SpringBootTest
@Testcontainers
class OrderIntegrationTest {

    @Container
    static PostgreSQLContainer postgres =
        new PostgreSQLContainer<>("postgres:16");

    @Container
    static KafkaContainer kafka =
        new KafkaContainer(
            DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));
}
```

6. Cenários de Teste — End-to-End

Validam o sistema completo em execução. Simulam o comportamento real do usuário e provam que todos os serviços funcionam juntos. São os testes mais importantes para a demonstração na banca.

ID	Cenário	Entrada	Resultado Esperado	Prioridade
CT-E-01	Fluxo completo de pedido	Cliente cria pedido via POST /api/orders	Pedido salvo no banco, evento no Kafka, WebSocket envia update ao frontend	Alta
CT-E-02	Atualização de status em tempo real	Admin atualiza status para 'Shipped' via PATCH /api/admin/orders/{id}/status	Frontend recebe atualização via WebSocket sem recarregar a página	Alta
CT-E-03	Hello World arquitetural	Clique no botão do frontend que chama POST /api/hello	Mensagem percorre Frontend → Logistics → Kafka → Notification → WebSocket	Alta
CT-E-04	Listagem de produtos na vitrine	GET /api/products retornado pelo Gateway na porta 8080	HTTP 200 com lista de produtos; Gateway roteou corretamente	Média
CT-E-05	Resiliência — notification-service reinicia	Derrubar notification-service, criar pedido, subir novamente	Pedido criado com sucesso; ao subir, consumer processa evento pendente	Média

Para a banca (CT-E-03): abrir o browser em localhost:5173, mostrar o badge 'WebSocket conectado', clicar no botão e mostrar a mensagem aparecendo em tempo real. Esse cenário sozinho prova toda a arquitetura funcionando.

7. Cenários de Teste — Não-Funcionais

Validam os Requisitos Não-Funcionais definidos na seção 2. Esses testes garantem que o sistema não apenas funciona corretamente, mas funciona bem.

7.1 Desempenho (RNF-01, RNF-02, RNF-03)

ID	Cenário	Parâmetros	Critério de Aceite	Ferramenta
CT-NF-01	Latência REST em carga	50 usuários, 60s, GET /api/products	P95 < 500ms	k6 ou JMeter
CT-NF-02	Latência ponta a ponta Kafka	Publicar 10 eventos consecutivos	Todos consumidos em < 2s	Log de timestamp
CT-NF-03	Throughput de pedidos	20 POST /api/orders por segundo	Zero erros HTTP 5xx	k6

7.2 Segurança (RNF-07, RNF-08, RNF-09)

ID	Cenário	Como executar	Critério de Aceite	Ferramenta
CT-NF-04	Credenciais no repositório	git log --all grep -i password	Zero ocorrências de senha no histórico Git	Manual
CT-NF-05	CORS — origem não autorizada	Requisição de http://malicious.com	HTTP 403 Forbidden	cURL / Postman
CT-NF-06	Validação de campo obrigatório	POST /api/orders sem campo customerEmail	HTTP 400 com mensagem 'email é obrigatório'	REST Assured

7.3 Portabilidade (RNF-13, RNF-14)

ID	Cenário	Como executar	Critério de Aceite	Ferramenta
CT-NF-07	Subir ambiente do zero	Máquina limpa: git clone + docker compose up -d	Todos containers 'running' em < 5 min	Docker
CT-NF-08	Independência de sistema operacional	Executar em Windows e macOS	Sistema funciona sem alterações	Manual

8. Ferramentas e Configuração

8.1 Stack de Testes

Ferramenta	Finalidade	Como incluir
JUnit 5	Framework principal de testes	Incluso no spring-boot-starter-test
Mockito	Criação de mocks para testes unitários	Incluso no spring-boot-starter-test
Testcontainers	PostgreSQL e Kafka reais nos testes	org.testcontainers:postgresql + kafka
Spring Boot Test	Context de teste com injeção de dependência	Incluso no spring-boot-starter-test
REST Assured	Testes de API REST de forma fluente	io.rest-assured:rest-assured
JaCoCo	Relatório de cobertura de código	Plugin Maven: jacoco-maven-plugin
k6	Testes de carga e desempenho	Instalação separada: k6.io

8.2 Dependências Maven (pom.xml de cada serviço)

```
org.springframework.boot
spring-boot-starter-test
test

org.testcontainers
postgresql
test

org.testcontainers
kafka
test
```

8.3 Configuração JaCoCo — Cobertura mínima 70%

```
org.jacoco
jacoco-maven-plugin

PACKAGE

LINE
COVEREDRATIO
0.70
```

Gerar relatório: ./mvnw verify → relatório em target/site/jacoco/index.html

9. Critérios de Aceite e Cobertura

Define o que precisa estar verde para o projeto ser considerado aprovado em qualidade, alinhado com os critérios de avaliação da N2.

9.1 Critérios mínimos para aprovação

Critério	Como verificar	Tipo
Testes unitários passando	100% dos testes CT-U-01 a CT-U-13 com status PASS	Obrigatório
Cobertura service layer $\geq 70\%$	Relatório JaCoCo com cobertura de linha $\geq 70\%$ no pacote service/	Obrigatório
Teste integração Kafka	CT-I-02 ou CT-I-03 provando consumo real de mensagem	Obrigatório
Fluxo E2E funcionando	CT-E-03 demonstrado ao vivo na banca	Obrigatório
RNFs documentados	Todos os 14 RNFs com critério de aceite definido neste documento	Obrigatório
Latência Kafka $< 2s$	CT-NF-02 executado e registrado em evidência (print de log)	Desejável
Zero credenciais no Git	CT-NF-04 executado e sem ocorrências	Obrigatório

9.2 Rastreabilidade — RNF × Critério de Avaliação

RNFs	Categoria	Critério de Avaliação	Impacto
RNF-01, 02, 03	Desempenho	Inovação e UI Final (0,3 N2)	Suporte
RNF-04, 05, 06	Confiabilidade	Mensageria desacoplada (0,4 N2)	Direto
RNF-07, 08, 09	Segurança	Qualidade de código (0,5 N2)	Direto
RNF-10, 11	Cobertura / Arq. Limpa	Qualidade e Testes (0,4 N2)	Direto
RNF-12	Config. externalizada	Organização (0,2 N1)	Direto
RNF-13, 14	Portabilidade	Docker da Fila/Broker (0,2 N1)	Direto

Resumo executivo: com os 13 cenários unitários, 6 de integração e 3 end-to-end passando, a cobertura JaCoCo acima de 70% e o CT-E-03 demonstrado ao vivo, o projeto atende todos os critérios de qualidade da N2 e está preparado para a arguição dos professores.